A higher-order function is a function that either takes another function as an argument or returns a function as its result. Higher-order functions can accept curried functions, enabling powerful composition and abstraction in code.

Together, currying and higher-order functions promote modular, composable, and maintainable code—hallmarks of functional programming in JavaScript. Listing 3.7 shows examples of currying and higher-order functions.

Listing 3.7 Examples of currying and higher-order functions

```
function multiply(a, b) { #A
  return a * b;
}
function curriedMultiply(a) { #B
  return function(b) {
    return a * b;
  };
}
const double = curriedMultiply(2); #C
console.log(double(5)); // 10
function withLogging(fn) { #D
  return function(...args) {
    console.log(`Calling with args: ${args}`);
    return fn(...args);
  };
}
const loggedDouble = withLogging(double); #E // Calling with args
loggedDouble(10);
```

For this topic, are you able to answer these follow-up/probing questions?

- "How would you write a generic curry utility that transforms any multi-argument function into a curried version?"
- "Can you implement your own version of `map` using only `reduce`?"
- "Are there performance implications of currying in JavaScript? How would you mitigate them in performance-critical code?"
- "What are some potential debugging challenges when working with deeply composed or curried functions, and how can they be addressed?"
- "How would you refactor a legacy codebase that uses imperative loops and callbacks to use curried and higher-order functions for improved readability and maintainability?"